

Universidad Rafael Landívar

Facultad de Ingeniería

Laboratorio de Microprogramación

Sección 9

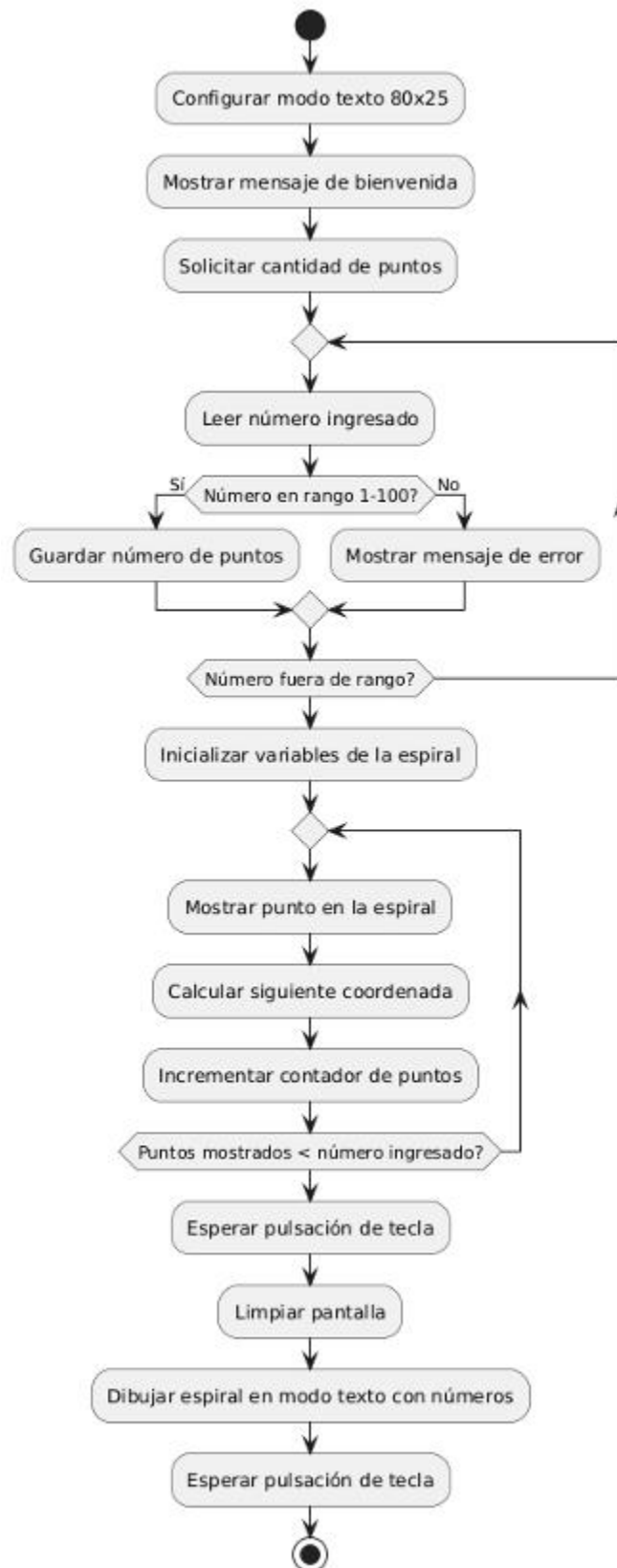
Proyecto#1 Microprogramación

Rodrigo Ruiz - 1037623

Anthony Hernández -1086223

Guatemala, 27 de marzo de 2025

Diseño de la solución:



Análisis sobre las distintas partes de la solución implementada

Sección Data

```
;-----  
; Proyecto de Microprogramacion  
;-----  
.MODEL SMALL  
.STACK 100h  
.DATA  
;-----  
; Todas las preguntas para el programa  
;-----  
; Mensajes  
mensajeBienvenida DB 'Espiral de Ulam - Proyecto de Microprogramacion$',0Dh,0Ah  
mensajePrompt DB 'Ingrese la cantidad de puntos (1-100): $'  
mensajeInvalido DB 0Dh,0Ah,'Error: Ingrese un numero entre 1 y 100.$'  
mensajeCoordenadas DB 0Dh,0Ah,'Coordenadas de la Espiral de Ulam:$',0Dh,0Ah  
mensajePresionarTecla DB 0Dh,0Ah,'Presione cualquier tecla para continuar...$',0Dh,0Ah  
  
; Variables de la espiral  
numeroPuntos DW 0 ; Número total de puntos solicitados  
numeroActual DW 0 ; Número actual en la espiral  
coordenadaX DW 0 ; Coordenada X actual (plano)  
coordenadaY DW 0 ; Coordenada Y actual  
direccion DB 0 ; 0=derecha, 1=arriba, 2=izquierda, 3=abajo  
longitudSegmento DW 0 ; Longitud del segmento actual  
segmentosRecorridos DW 0 ; Pasos recorridos en la dirección actual  
puntosMostrados DW 0 ; Contador de puntos ya mostrados  
  
; Buffer para conversión de números a cadena  
bufferNumero DB 6 DUP ('$')  
valor10 DW 10  
  
; Constantes para centrar la espiral en modo texto (80x25)  
CENTRO_PANTALLA_COL EQU 40  
CENTRO_PANTALLA_ROW EQU 12  
.CODE  
;-----  
; PROCEDIMIENTO PRINCIPAL  
;-----
```

1. Define los mensajes, variables y constantes usadas en el programa:
2. Mensajes: Textos para interacciones con el usuario
3. Variables de la espiral: Almacenan datos clave como la cantidad de puntos, coordenadas, dirección y longitud de segmentos.
4. Buffer de conversión: Espacio reservado para convertir números a texto.
5. Constantes de pantalla: Posición central de la espiral en una pantalla de 80x25 caracteres.

Sección Principal

```
;-----  
; PROCEDIMIENTO PRINCIPAL  
;-----  
PRINCIPAL PROC  
    mov ax, @DATA  
    mov ds, ax  
  
    ; Configurar modo texto (80x25)  
    mov ah, 0  
    mov al, 3  
    int 10h  
  
    ; Mostrar mensaje de bienvenida  
    mov ah, 09h  
    lea dx, mensajeBienvenida  
    int 21h  
    call SaltoLinea  
  
    ; Obtener número de puntos  
ObtenerPuntos:  
    mov ah, 09h  
    lea dx, mensajePrompt  
    int 21h  
  
    call LeerNumero  
    mov numeroPuntos, ax  
  
    ; Validar rango 1-100  
    cmp ax, 1  
    jl ErrorEntrada  
    cmp ax, 100  
    jg ErrorEntrada  
    jmp EntradaCorrecta  
  
    ; Programacion defensiva  
ErrorEntrada:  
    call SaltoLinea  
    mov ah, 09h  
    lea dx, mensajeInvalido  
    int 21h  
    jmp ObtenerPuntos
```

EntradaCorrecta:

```
; Inicializar variables de la espiral para mostrar coordenadas
mov numeroActual, 1
mov coordenadaX, 0
mov coordenadaY, 0
mov direccion, 0
mov longitudSegmento, 1
mov segmentosRecorridos, 0
mov puntosMostrados, 0

; Mostrar las coordenadas (para ver el recorrido calculado)
call SaltoLinea
mov ah, 09h
lea dx, mensajeCoordenadas
int 21h
call SaltoLinea
```

MostrarCoordenadas:

```
mov ax, puntosMostrados
cmp ax, numeroPuntos
jae FinMostrarCoordenadas
call MostrarPunto
call SiguientePunto
inc puntosMostrados
jmp MostrarCoordenadas
```

FinMostrarCoordenadas:

```
; Esperar pulsación
call EsperarTecla

; Limpiar pantalla
call ClearScreen

; Dibujar la espiral en modo texto con números
call DibujarEspiralNumeros

; Esperar antes de salir
call EsperarTecla

mov ah, 4Ch
int 21h
```

PRINCIPAL ENDP

1. Configura el modo de texto en 80x25.
2. Muestra el mensaje de bienvenida.
3. Solicita al usuario la cantidad de puntos (1-100) y valida la entrada.

4. Inicializa variables de la espiral.
5. Muestra las coordenadas calculadas.
6. Espera a que el usuario presione una tecla.
7. Limpia la pantalla.
8. Dibuja la espiral con números y asteriscos.
9. Espera otra tecla antes de finalizar.

Sección Utilidad

```
;-----  
; PROCEDIMIENTOS DE UTILIDAD  
;-----  
SaltoLinea PROC  
    mov ah, 02h  
    mov dl, 0Dh  
    int 21h  
    mov dl, 0Ah  
    int 21h  
    ret  
SaltoLinea ENDP  
  
EsperarTecla PROC  
    call SaltoLinea  
    mov ah, 09h  
    lea dx, mensajePresionarTecla  
    int 21h  
    mov ah, 00h  
    int 16h  
    ret  
EsperarTecla ENDP  
  
LeerNumero PROC  
    push bx  
    xor bx, bx           ; Resultado en BX  
  
LecturaLoop:  
    mov ah, 01h  
    int 21h  
    cmp al, 0Dh         ; ¿Enter?  
    je FinLectura  
    cmp al, '0'  
    jb LecturaLoop  
    cmp al, '9'  
    ja LecturaLoop  
    sub al, '0'  
    mov cl, al  
    mov ax, bx  
    mul valor10  
    add ax, cx  
    mov bx, ax  
    jmp LecturaLoop
```

FinLectura:

```
mov ax, bx
pop bx
ret
```

LeerNumero ENDP

ConvertirNumeroACadena PROC

```
push ax
push bx
push si
```

```
lea si, bufferNumero
mov bx, 10
xor cx, cx
```

```
test ax, ax
jns Positivo
neg ax
mov byte ptr [si], '-'
inc si
```

Positivo:

```
xor dx, dx
div bx
add dl, '0'
push dx
inc cx
test ax, ax
jnz Positivo
```

Desapilar:

```
pop dx
mov [si], dl
inc si
loop Desapilar
```

```
mov byte ptr [si], '$'
pop si
pop bx
pop ax
ret
```

ConvertirNumeroACadena ENDP

MostrarPunto PROC

```
    push ax
    push bx

    ; Imprimir número de espiral
    mov ax, numeroActual
    call ConvertirNumeroACadena
    mov ah, 09h
    lea dx, bufferNumero
    int 21h

    ; Imprimir ". (X,Y)"
    mov ah, 02h
    mov dl, '.'
    int 21h
    mov dl, ' '
    int 21h
    mov dl, '('
    int 21h

    ; Imprimir coordenadaX
    mov ax, coordenadaX
    call ConvertirNumeroACadena
    mov ah, 09h
    lea dx, bufferNumero
    int 21h

    ; Imprimir coma
    mov ah, 02h
    mov dl, ','
    int 21h

    ; Imprimir coordenadaY
    mov ax, coordenadaY
    call ConvertirNumeroACadena
    mov ah, 09h
    lea dx, bufferNumero
    int 21h

    ; Cerrar paréntesis y salto de línea
    mov ah, 02h
    mov dl, ')'
    int 21h
    call SaltoLinea
```

```
pop bx
pop ax
ret
```

MostrarPunto ENDP

1. Imprime un salto de línea.
2. Muestra un mensaje y espera a que el usuario presione una tecla.
3. Lee un número ingresado por el usuario, validando solo caracteres numéricos.
4. Convierte un número en su representación de texto.
5. Imprime el número actual junto con sus coordenadas.

SiguientePunto PROC

```
push ax
mov al, direccion
cmp al, 0
je Derecha
cmp al, 1
je Arriba
cmp al, 2
je Izquierda
jmp Abajo
```

Derecha:

```
inc coordenadaX
jmp ActualizarContador
```

Arriba:

```
dec coordenadaY
jmp ActualizarContador
```

Izquierda:

```
dec coordenadaX
jmp ActualizarContador
```

Abajo:

```
inc coordenadaY
```

ActualizarContador:

```
inc segmentosRecorridos
mov ax, segmentosRecorridos
cmp ax, longitudSegmento
jb FinMovimiento
```

```
mov segmentosRecorridos, 0
inc direccion
mov al, direccion
cmp al, 4
jb NoReset
mov direccion, 0
```

NoReset:

```
test direccion, 1
jnz FinMovimiento
inc longitudSegmento
```

```
FinMovimiento:  
    inc numeroActual  
    pop ax  
    ret  
SiguientePunto ENDP
```

1. Mueve el punto en la dirección actual (derecha, arriba, izquierda, abajo).
2. Controla los cambios de dirección cuando se completan los segmentos.
3. Ajusta la longitud de los segmentos después de dos giros.

Sección Dibujar un número y asteriscos

DibujarNumeroYAsterisco PROC

```
    push ax
    push bx
    push dx
    push cx

    ; Guardar número actual para usarlo después
    mov cx, numeroActual

    ; Calcular columna: centro + coordenadaX * 3 (para dar más espacio)
    mov ax, coordenadaX
    mov bx, 3                ; Factor de escala horizontal
    imul bx
    add ax, CENTRO_PANTALLA_COL
    mov dl, al                ; DL = columna

    ; Calcular fila: centro - coordenadaY (invierte Y para que crezca hacia arriba)
    mov ax, CENTRO_PANTALLA_ROW
    sub ax, coordenadaY
    mov dh, al                ; DH = fila

    ; Verificar límites de pantalla
    cmp dl, 79
    jg FueraDePantalla
    cmp dh, 24
    jg FueraDePantalla
    cmp dl, 0
    jl FueraDePantalla
    cmp dh, 0
    jl FueraDePantalla

    ; Posicionar el cursor
    mov ah, 02h
    mov bh, 0                ; Página 0
    int 10h

    ; Mostrar el número
    mov ax, cx                ; Recuperar número actual
    call ConvertirNumeroACadena
    mov ah, 09h
    lea dx, bufferNumero
    int 21h
```

```

; Posicionar el cursor para el asterisco (siguiente posición)
inc dl
mov ah, 02h
mov bh, 0
int 10h

; Mostrar asterisco después del número si no es el último
mov ax, cx          ; Recuperar número actual
cmp ax, numeroPuntos
je OmitirAsterisco

; Mostrar asterisco
mov ah, 02h
mov dl, '*'
int 21h

OmitirAsterisco:
FueraDePantalla:
    pop cx
    pop dx
    pop bx
    pop ax
    ret
DibujarNumeroYAsterisco ENDP

DibujarEspiralNumeros PROC
; Limpiar pantalla nuevamente para asegurar un lienzo limpio
call ClearScreen

; Reinicializar las variables de la espiral para el dibujo
mov numeroActual, 1
mov coordenadaX, 0
mov coordenadaY, 0
mov direccion, 0
mov longitudSegmento, 1
mov segmentosRecorridos, 0
mov puntosMostrados, 0

; Dibujar el primer punto (número 1 en el centro)
call DibujarNumeroYAsterisco
inc puntosMostrados

LoopDibujo:
; Verificar si ya mostramos todos los puntos solicitados
mov ax, puntosMostrados
cmp ax, numeroPuntos
jae FinDibujarEspiral

; Calcular siguiente posición y dibujar
call SiguientePunto
call DibujarNumeroYAsterisco
inc puntosMostrados
jmp LoopDibujo

FinDibujarEspiral:
    ret
DibujarEspiralNumeros ENDP

```

1. Reinicializa variables de la espiral.
2. Dibuja cada número en su posición correcta en pantalla.
3. Controla el desplazamiento y asegura que los caracteres no salgan de los límites de la pantalla.

```

;-----
; PROCEDIMIENTO PARA LIMPIAR LA PANTALLA (MODO TEXTO)
;-----
ClearScreen PROC
    mov ah, 06h
    mov al, 0
    mov bh, 07h      ; Atributo de fondo/texto
    mov cx, 0
    mov dx, 184Fh    ; (80*25)-1
    int 10h

    ; Resetear la posición del cursor
    mov ah, 02h
    mov bh, 0
    mov dx, 0
    int 10h
    ret
ClearScreen ENDP

END PRINCIPAL

```

Borra el contenido de la pantalla y reinicia la posición del cursor.